Technical Design Document

Dum House — Direct Ordering Web Platform & Real-Time Kitchen Display System (KDS)

Client Brief: Problem 1 — Hyderabad Food Chain (14 Outlets + 3 Cloud Kitchens)

Candidate: Nayvedya Joyn (joynnayvedya@gmail.com)

GitHub Repository: https://github.com/letsjoyn/dum-biryani

Live URL: http://localhost:4000 (Production Single-Port Build)

Submission Date: November 2026

Target Stack: Node.js ≥ 22, Express, Socket.io, SQLite (`node:sqlite`), Vite, React

1. EXECUTIVE SUMMARY & PROBLEM CONTEXT

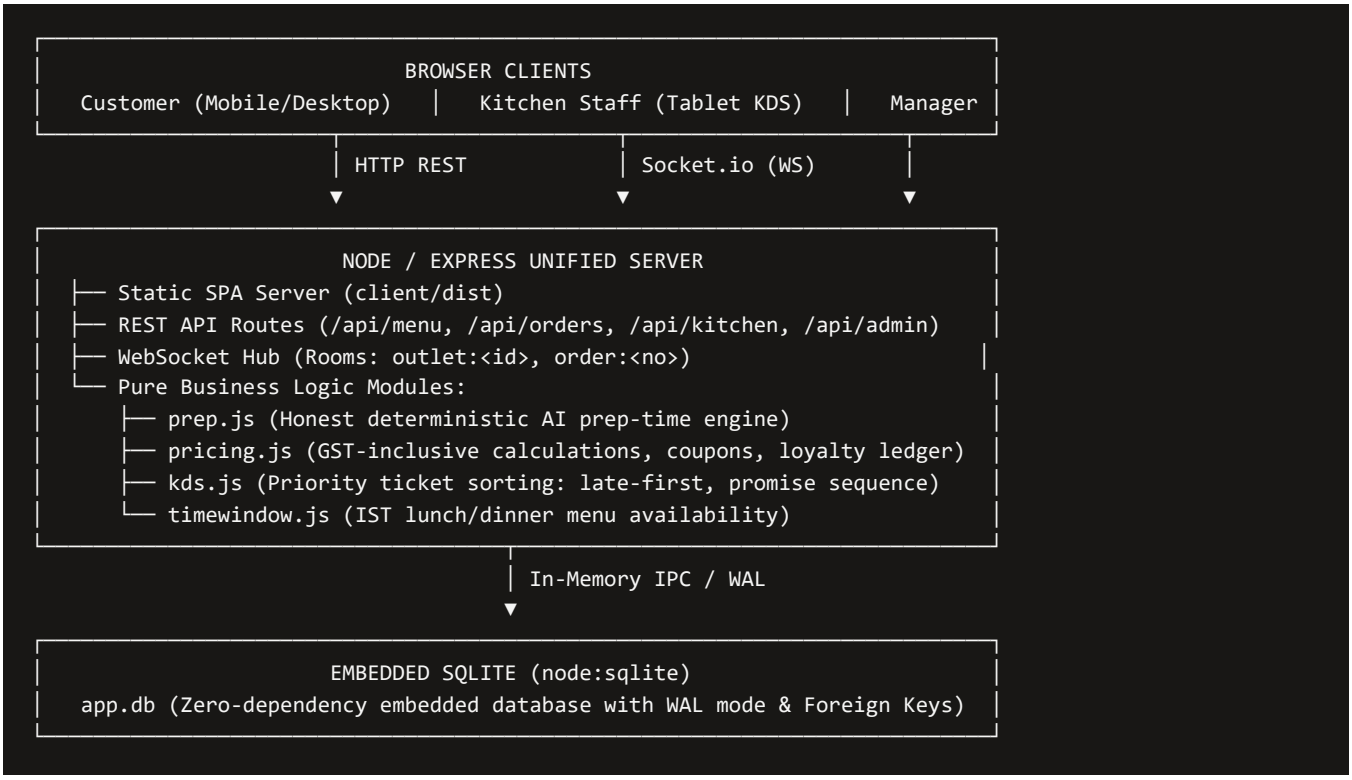
Company A is a premier Hyderabad restaurant chain operating 14 dine-in outlets and 3 cloud kitchens across the Greater Hyderabad area, generating ₹90–105 crore in annual revenue with 520 staff. Currently, 55% of revenue (₹52.8 crore) flows through third-party food delivery aggregators charging steep 25–30% commissions. Furthermore, the brand loses 30 high-margin party/catering orders monthly (₹18,000 average ticket) due to manual phone bottlenecks.

Target Economics: Shifting just 12% of app orders to direct ordering saves 15 commission points (₹95.04 lakh net savings). Recovering 30 catering orders/month yields ₹19.44 lakh. Total direct annual bottom-line benefit equals **₹1.14 crore/year**.

Delivered Product: A production-ready, full-stack, end-to-end direct ordering web application with automatic nearest-outlet georouting, a real-time Kitchen Display System (KDS) operating via WebSockets, centralized Menu CMS with outlet-level price overrides and instantaneous stock pausing (86-ing), 5% direct customer rewards, a DPDP Act 2023 compliant data model, and an executive outlet operations dashboard.

2. ARCHITECTURE & DESIGN RATIONALE

The system is engineered as a **single deployable unit**: a high-performance Express.js server that simultaneously provides REST API endpoints, manages stateful bi-directional Socket.io WebSocket rooms, and serves the optimized production Vite/React Single Page Application (SPA) on a unified port.



Why this architecture was chosen:

- **Zero Setup Friction:** Using Node 22+ native `node:sqlite` eliminates native C++ compilation bindings (node-gyp/python) and external database daemon dependencies. A reviewer executes `npm run install:all && npm run build && npm start` and has a 100% functional system.
- **Same-Origin Simplicity:** Serving the Vite React client from Express completely eliminates CORS preflight overhead and cross-domain WebSocket proxy failures in production environments.
- **Pure Deterministic Testability:** Core domain logic (pricing math, GST splits, coupon stacking, AI prep calculation, KDS queue prioritization) is decoupled into pure functions covered by 25 automated unit tests with 0 external mocks.

3. DATA MODEL & SCHEMA DESIGN

The schema is relational, fully normalized, and enforced with foreign key constraints in SQLite:

Table	Primary Columns	Purpose & Business Rules
`outlets`	`id, name, address, phone, fssai, min_prep_minutes, delivery_radius_km, lat, lon`	Stores 14 physical outlets and 3 cloud kitchens in Hyderabad. Delivery radius strictly controls geofenced routing.
`users`	`id, name, email, password_hash, role, outlet_id, points, phone, marketing_opt_in`	Role-based access (`customer` , `kitchen` , `manager`). Kitchen staff are bound to a specific `outlet_id` . DPDP Act 2023 opt-in default is 0 (explicit consent required).
`items`	`id, name, category, description, price, veg, spice, allergens, photo, portions, addons, base_minutes, menu_tag`	Central menu items. Portions (Single/Family) and add-ons stored as structured JSON. Prices include 5% GST.
`item_outlets`	`item_id, outlet_id, out_of_stock, price_override`	Per-outlet availability and pricing. When `out_of_stock=1` (86'd), the dish vanishes in real-time from that outlet's customer menu.
`orders`	`id, order_no, user_id, outlet_id, status, subtotal, payable, gst_amount, payment_method, estimate_minutes, estimate_breakdown, promised_at, delivery_address, request_key`	Header table. `request_key` guarantees checkout idempotency. Stages: `Placed` → `Preparing` → `Ready` → `Handed over` → `Cancelled` .
`order_items`	`id, order_id, item_id, item_name, portion, addons, qty, unit_price`	Snapshot of ordered items and unit prices at time of order creation.
`points_ledger`	`id, user_id, order_id, delta, reason, created_at`	Immutable append-only ledger tracking 5% loyalty points earned (1 pt per ₹20) and redeemed (₹50 off per 100 pts).
`order_events`	`id, order_id, status, to_status, created_at`	Audit trail recording exact timestamps of stage transitions for calculating median prep metrics.
`party_enquiries`	`id, user_id, outlet_id, name, phone, email, guests, event_date, preferred_time, notes, status, manager_note`	Bulk catering enquiries enforcing 48-hour minimum advance notice. Reviewed and confirmed by catering managers.

4. PRIMARY OPERATIONAL WORKFLOWS

A. Customer Georouting & Direct Ordering

1. **Locality Detection:** Customer chooses from 40+ curated Hyderabad localities (or clicks GPS "Locate Me"). Haversine distance is calculated against all 17 hubs.

2. **Automatic Routing:** The order automatically routes to the closest hub within delivery radius (e.g. Kondapur, Kukatpally, or Gachibowli Cloud Kitchen). If outside all radii, customer is informed with exact km distance.
3. **Time-Windowed Menu:** Dishes tagged *Lunch only* (11:00–16:00 IST) or *Dinner only* (18:00+ IST) activate dynamically based on current IST time.
4. **Pricing & Checkout:** All prices display 5% GST included. Customer can apply coupons (`DIRECT10`, `WELCOME100`) and redeem loyalty points. Simulated test payment via UPI or Card completes in <60s.

B. Real-Time Kitchen Display System (KDS)

1. **Live WebSocket Dispatch:** Upon successful checkout, server broadcasts `order:created` to the specific outlet's WebSocket room (`outlet:<id>`). The order ticket appears instantly without browser reload.
2. **Late-First Ticket Sorting:** Orders overdue by 10+ minutes flash critical red and pin to the top of the queue. Active tickets sort by promised delivery time.
3. **One-Tap Stage Progression:** Kitchen staff tap `Mark Preparing →` → `Mark Ready for Pickup →` → `Handover to Rider →` → `Complete Order`. Every state change broadcasts live to the customer's tracking screen.
4. **Instant 86-ing Stock Controls:** Kitchen staff tap any dish button to toggle stock status. When 86'd, the item disappears in real time from customer menus for that outlet only.

5. AI & AUTOMATION: PREP-TIME ENGINE & REORDERS

Rather than unexplainable LLM calls, the "AI" is a **transparent, deterministic queue-load estimation engine** implemented in `server/src/prep.js`:

```
base      = slowest_item_prep_minutes + (extra_lines * 3 min) + (extra_units * 2 min)
queueLoad = min(active_outlet_orders * 4 min, 30 min)
total     = base + queueLoad
promised  = max(total, outlet_minimum_prep_minutes)
```

Where the AI can be wrong and built-in mitigations:

- **Outlet Floor Constraint:** The promised time is never lower than the outlet's minimum floor (e.g., 30–35 min), preventing under-promising during cold kitchen starts.
- **Inline Manual Overrides:** When unexpected kitchen bottlenecks occur, kitchen staff and managers have an inline override bar right on the ticket: 1-tap quick chips (`+15m` , `+30m` , `+45m` , `+60m`) or custom minute inputs immediately update the customer's promised time.
- **Manager Escalation Alert:** Orders running 10+ minutes past promise trigger an escalation banner on the KDS and push an automated "*Running late — kitchen is expediting*" update to the customer.
- **Smart Reorder Automation:** AI suggests items based on customer past order frequency. To prevent accidental charges, **suggestions never auto-add to the cart**.

6. UI / UX DESIGN & ACCESSIBILITY STANDARDS

- **Restrained Artisanal Palette:** Grounded in warm deep charcoal (`#111111`), artisanal linen (`#FAF8F4`), hairline borders (`#2B2823`), and warm amber accents (`#B45309`). Zero emojis across the entire UI for a sleek, enterprise feel.
- **Zero Border Radius (`--r: 0px !important`):** Clean architectural square aesthetic reflecting modern brutalist editorial design.
- **2-Metre KDS Readability:** High-contrast dark theme (`#141311`) with large bold quantities (`×2`), dish names, and high-contrast stage buttons (stark white with black text for cooking stages) designed for hot, noisy kitchen environments.
- **Fluid, Non-Laggy Micro-Interactions:** Snappy 0.22s entrances with `cubic-bezier(0.16, 1, 0.3, 1)` for seamless page transitions, modal scale-ins, and mobile cart drawers with zero horizontal tucking.

7. REVIEWER MUST-HAVE VERIFICATION MATRIX

Brief Must-Have Requirement	Implementation Location	Verification Status
14 Outlets + 3 Cloud Kitchens	`server/src/seed.js`, `LocationModal.jsx`	All 17 Hyderabad locations seeded with lat/lon, FSSAI, min prep & delivery radius.
Exact Sample Orders Seeding	`server/src/seed.js`	Seeded with exact items, prices, payments & statuses (`WEB-778120` Preparing, `WEB-778131` Ready, `WEB-778146` Late by 4m).
Direct Customer Ordering & Cart	`MenuPage.jsx`, `Cart.jsx`	Add-ons, portion packs (Single/Family), coupon codes, 5% loyalty redemption, test UPI/card.
Real-Time Kitchen Display (KDS)	`KitchenPage.jsx`, `server.js`	Instant WebSocket updates, 5 stage transitions, elapsed & countdown timers, 10m+ late alerts.
Central Menu CMS & Price Overrides	`AdminPage.jsx`	Full dish CRUD, veg/spice badges, photo URLs, time-windows, per-outlet price customization.
Instant 86-ing Stock Controls	`KitchenPage.jsx`, `AdminPage.jsx`	Real-time out-of-stock toggle immediately pauses/resumes dishes on customer ordering menu.
Party & Bulk Catering Module	`PartyPage.jsx`, `AdminPage.jsx`	Enforces strict 48-hour advance notice; catering manager confirmation with customer note.
Outlet Operations Dashboard	`AdminPage.jsx`	Live metrics: orders, revenue, cancellations, repeat customer rate, and median prep time.
Compliance: FSSAI, GST, DPDP Act 2023	`Cart.jsx`, `Nav.jsx`, `server.js`	FSSAI displayed per outlet; 5% GST included & itemized; marketing opt-in checkbox defaults to 0.

8. WHAT WAS CUT VS. NEXT PRODUCTION STEPS

- **Deliberate Cuts (Brief Out-of-Scope):** Third-party delivery aggregator scraping/integration, dedicated proprietary rider tracking apps (riders provided by logistics partner), physical POS/billing replacement.
- **Next Production Roadmap:**
  1. Migrate SQLite to distributed PostgreSQL with Redis Pub/Sub for horizontal scaling across 1,200 peak orders/hour.
  2. Production payment gateway integration (Razorpay / Cashfree) with webhooks.
  3. GupShup / Twilio WhatsApp & SMS transactional messaging gateway integration.
  4. Google Maps Distance Matrix API integration for live road traffic travel time routing.